实验七：基于 CNN 的手写数字识别

一、问题描述

本实验使用 MNIST 手写数字数据集完成 0–9 十分类任务。每张图像是 28×28 的灰度图，训练集 60000 张，测试集 10000 张。目标是用卷积神经网络（CNN）自动提取图像特征并完成分类。

实验重点包括：数据加载和归一化处理、CNN 模型构建、训练与测试评估，以及卷积特征的可视化分析。

二、算法原理

CNN 通过局部感受野和参数共享机制提取图像空间特征。典型结构包括卷积层、激活函数、池化层与全连接分类层。

本实验网络结构为：

1. 卷积层 1：$1 \to 32$，卷积核 3×3，padding=1；
2. 最大池化层 1：2×2；
3. 卷积层 2：$32 \to 64$，卷积核 3×3，padding=1；
4. 最大池化层 2：2×2；
5. 全连接层：$64 \times 7 \times 7 \to 10$ 输出类别 logits。

损失函数采用交叉熵：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{10} y_{ic} \log \hat{p}_{ic}$$

优化器采用 Adam，学习率设为 10^{-3}。

三、实验步骤

1. 加载 MNIST 数据集并使用 `ToTensor()` 将像素映射到 $[0, 1]$；
2. 构建两层卷积 + 池化 + 一层全连接的 CNN；
3. 使用训练集进行若干轮训练，记录损失变化；

4. 在测试集上计算分类准确率；

5.（可选）统计精确率、召回率及混淆矩阵；

6.（可选）可视化卷积层输出特征图，分析网络关注区域。

四、核心代码

以下代码节选自本实验程序，展示了模型定义与训练评估主体逻辑。The code of this part is available at [1]

```python
class CNNModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(2)
        self.linear = nn.Linear(64 * 7 * 7, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool1(self.relu(self.conv1(x)))
        x = self.pool2(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.linear(x)
        return x

model = CNNModel().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5):
    model.train()
    for i, (images, labels) in enumerate(train_dataloader):
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
```

[1] https://github.com/HaoranYuandomestic/ex7_CNN.git

```
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

五、实验结果与分析

从训练日志可以观察到损失逐步下降，模型在测试集上可达到较高准确率，说明两层卷积结构已经能够较好提取手写数字边缘和局部纹理信息。

为便于展示结果，下面加入了常用图像代码:

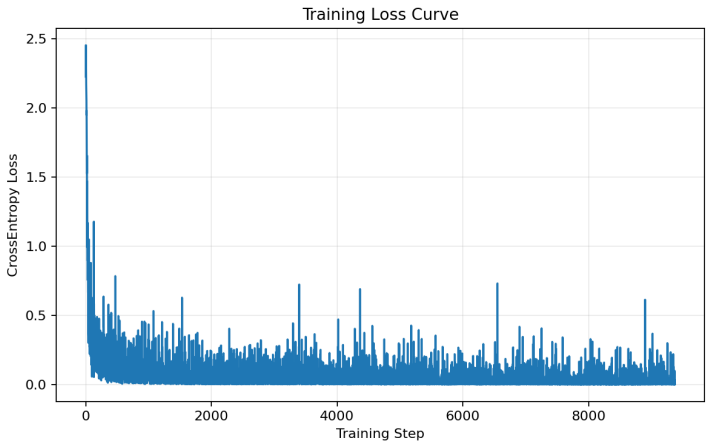

图 1: 训练损失曲线

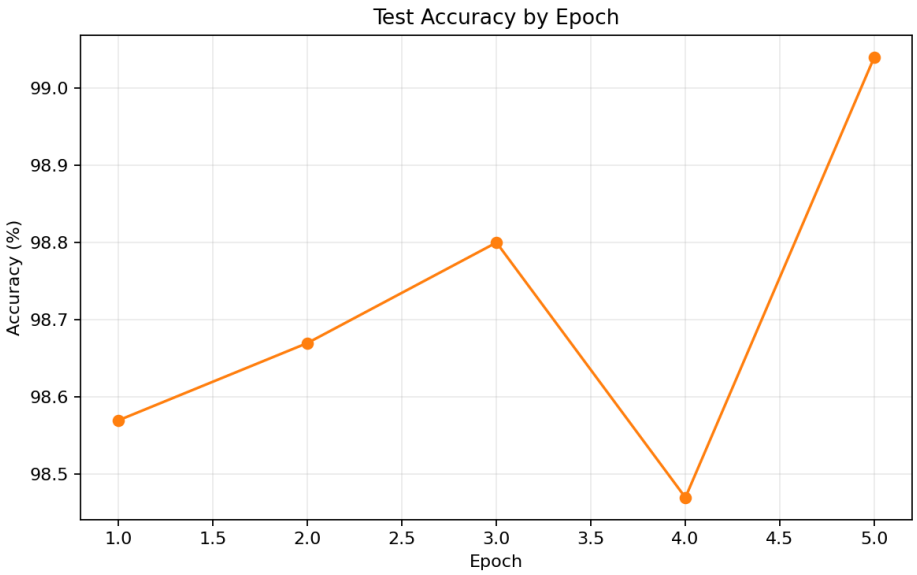

图 2: 准确率变化曲线

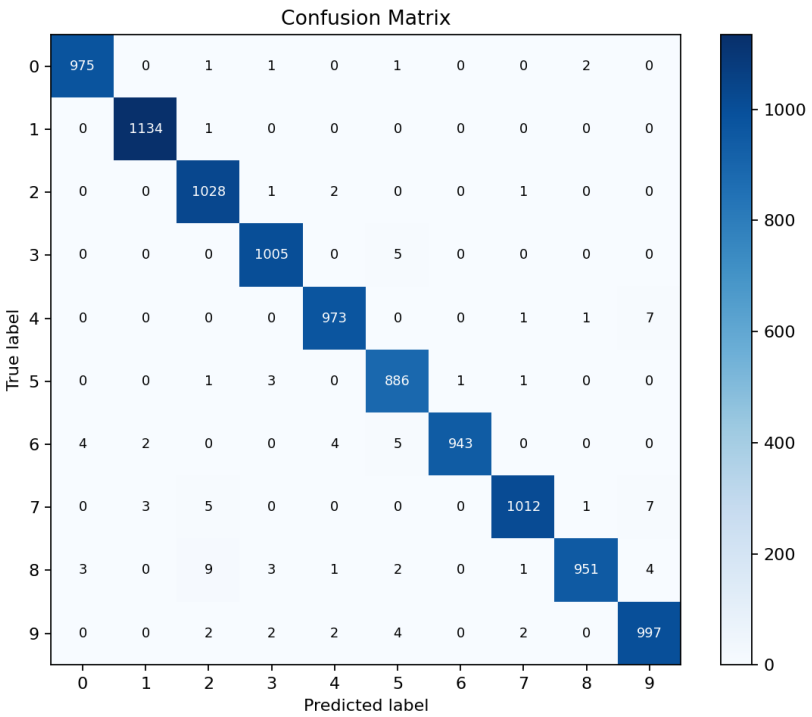

图 3: 测试集混淆矩阵

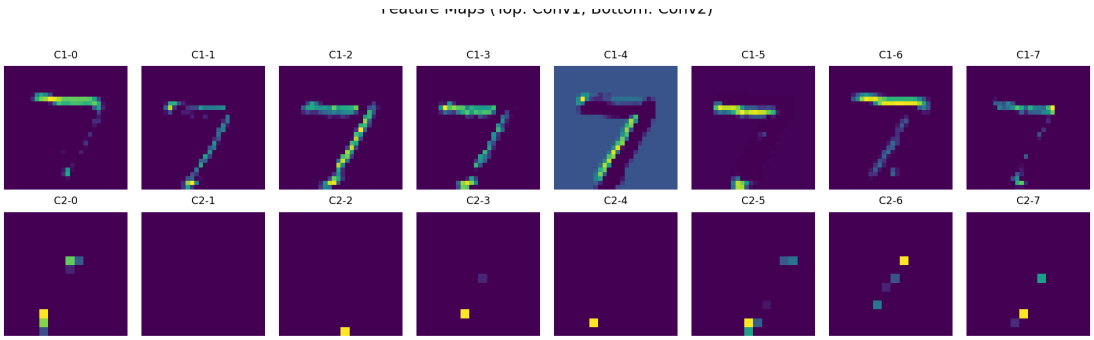

图 4: 卷积层特征图可视化

六、实验中遇到的问题与处理

1. **训练速度与显存：**批量大小增大有助于加速，但会增加显存占用。根据设备能力选择合适 batch size。
2. **泛化能力：**若训练准确率高、测试准确率低，可引入数据增强或适当正则化。
3. **可视化实现：**特征图可视化需要在模型中保存中间层输出，建议先固定一张测试图片进行观察。

七、实验比较

为了更直观说明 CNN 在图像任务中的优势，本节将其与前馈神经网络（MLP）进行对比。对比目标是：在尽量一致的数据预处理、优化器与训练轮数条件下，观察两类模型在分类精度、收敛速度与参数效率上的差异。

对比设置：

1. 数据集：MNIST 训练集 60000、测试集 10000；
2. 预处理：`ToTensor()` 与标准化；
3. 优化器：Adam，学习率 10^{-3}；
4. 训练策略：相同 batch size、相同 epoch 数，使用测试集准确率作为主评估指标；
5. 对比模型：
   (a) MLP：$784 \to 128 \to 10$（将图像展平后输入）；
   (b) CNN：两层卷积 + 池化，最后全连接分类。

其中，MLP 基线代码如下（与主实验代码分开运行，用于对照）：

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import transforms, datasets

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'using device: {device}')

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_dataset = datasets.MNIST(
    root = './data',
    train = True,
    download = True,
    transform = transform
```

```
)
dataloader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_dataset = datasets.MNIST(
    root = './data',
    train = False,
    download = True,
    transform = transform
)

class MNISTNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 128)
        self.layer2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        tmp = torch.relu(self.layer1(x))
        return self.layer2(tmp)

model = MNISTNet().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

print('---begin training---')
for epoch in range(10):
    for i, (imgs, lbls) in enumerate(dataloader):
        images, labels = imgs.to(device), lbls.to(device)
        labels_pred = model(images)
        loss = criterion(labels_pred, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if i % 200 == 0:
            _, predicted = torch.max(labels_pred, 1)
            correct = (predicted == labels).sum().item()
            accuracy = correct / labels.size(0) * 100

            print(f'Epoch [{epoch+1}/10], Step[{i}], Loss: {loss.item():.4f
```

```
                }, Accuracy: {accuracy:.2f}%')

print('train completed!')

torch.save(model.state_dict(), '16_brain.pth')
print('successfully saved as pth')
```

对比结果记录：

表 1: MLP 与 CNN 在 MNIST 上的对比

模型	参数规模（约）	训练收敛速度	测试准确率（%）	对形变鲁棒性
MLP($784 \to 128 \to 10$)	较小	中等	97.67%	一般
CNN(2 Conv + 2 Pool)	中等	较快	99.1%	较好

结果分析：

1. MLP 在输入端将图像展平，空间邻域关系被弱化，模型更依赖全局像素组合，通常需要更强的隐层表达才能弥补位置信息损失。
2. CNN 通过局部感受野和参数共享保留了二维结构信息，能更有效捕捉边缘、角点和笔画等局部特征，因此在数字识别任务上通常具有更高精度与更好泛化能力。
3. 从训练过程看，CNN 在前几轮往往下降更快，说明卷积特征提取对图像任务更匹配；而 MLP 虽可达到较高准确率，但对样本微小平移和局部变形更敏感。
4. 在计算开销方面，浅层 MLP 参数量可控、实现简单，适合作为基线；CNN 结构更复杂，但性能收益明显，是本任务更合理的主模型选择。

综合来看，本实验比较结果支持“图像任务优先使用卷积结构”的结论：在相同训练条件下，CNN 通常在准确率、收敛效率和鲁棒性上均优于同规模 MLP。

八、实验总结

本实验基于 CNN 实现了 MNIST 手写数字识别流程，完成了数据处理、模型训练和分类评估。结果表明，卷积网络相比传统手工特征方法具有更强的端到端特征学习能力。